

Infix to Prefix conversion and the Evaluation

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

#define MAX 200

// Operator stack (for conversion)
char opStack[MAX];
int opTop = -1;

// Evaluation stack (for prefix evaluation)
double evalStack[MAX];
int evalTop = -1;

// ---- operator stack functions ----
void pushOp(char c) {
    if (opTop < MAX - 1) opStack[++opTop] = c;
    else {
        printf("Error: Operator stack overflow.\n");
        exit(1);
    }
}
```

```

    }
}

char popOp() {
    if (opTop == -1) return '\0';
    return opStack[opTop--];
}

char peekOp() {
    if (opTop == -1) return '\0';
    return opStack[opTop];
}

// ---- eval stack functions ----

void pushEval(double v) {
    if (evalTop < MAX - 1) evalStack[++evalTop] = v;
    else {
        printf("Error: Evaluation stack overflow.\n");
        exit(1);
    }
}

double popEval() {
    if (evalTop == -1) {
        printf("Error: Evaluation stack underflow.\n");
        exit(1);
    }
    return evalStack[evalTop--];
}

```

```

// precedence & associativity
int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int isRightAssociative(char op) {
    return (op == '^');
}

int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' ||
c == '^');
}

// reverse string in-place
void reverseStr(char *s) {
    int n = strlen(s);
    for (int i = 0; i < n / 2; ++i) {
        char t = s[i];
        s[i] = s[n - i - 1];
        s[n - i - 1] = t;
    }
}

// swap brackets after reverse
void swapBrackets(char *s) {
    for (int i = 0; s[i]; ++i) {

```

```

        if (s[i] == '(') s[i] = ')';
        else if (s[i] == ')') s[i] = '(';
    }
}

/*
 * Infix -> Prefix
 */
void infixToPrefix(const char infix[], char prefix[]) {
    char expr[MAX];

    char temp[MAX * 2]; // Needs more space for multi-digit
    numbers

    int k = 0;

    strncpy(expr, infix, MAX - 1);
    expr[MAX - 1] = '\0';

    reverseStr(expr);
    swapBrackets(expr);

    opTop = -1;
    k = 0;
    int i = 0;

    while (expr[i] != '\0') {
        char ch = expr[i];

        if (isspace(ch)) {

```

```

        ++i;
        continue;
    }

    if (isdigit(ch)) {
        // Read multi-digit number
        int j = i;
        while (isdigit(expr[j])) {
            temp[k++] = expr[j++];
        }
        temp[k++] = ' '; // Add space separator
        i = j;
        continue; // Go to next character after the
number
    } else if (isalpha(ch)) {
        temp[k++] = ch;
        temp[k++] = ' ';
    } else if (ch == '(') {
        pushOp(ch);
    } else if (ch == ')') {
        while (opTop != -1 && peekOp() != '(') {
            temp[k++] = popOp();
            temp[k++] = ' ';
        }
        if (peekOp() == '(') popOp();
    } else if (isOperator(ch)) {
        // Handle negative numbers or signs

```

```

        // A minus sign is a unary operator if it
follows '(', or another operator

        if (ch == '-' && (i == 0 || expr[i-1] == '(' ||
isOperator(expr[i-1]))) {

            temp[k++] = '#'; // Use a special marker
for unary minus

            ++i;

            continue;

        }

        while (opTop != -1 && isOperator(peekOp()) &&
                ((precedence(peekOp()) > precedence(ch))
||
                (precedence(peekOp()) == precedence(ch)
&& !isRightAssociative(ch)))) {

            temp[k++] = popOp();

            temp[k++] = ' ';

        }

        pushOp(ch);

    }

    ++i;

}

while (opTop != -1) {

    temp[k++] = popOp();

    temp[k++] = ' ';

}

temp[k] = '\\0';

reverseStr(temp);

```

```

// Remove trailing spaces
while (temp[strlen(temp) - 1] == ' ') {
    temp[strlen(temp) - 1] = '\0';
}

strncpy(prefix, temp, MAX * 2 - 1);
prefix[MAX * 2 - 1] = '\0';
}

/*
 * Evaluate prefix expression
 */
double evaluatePrefix(const char prefix[]) {
    evalTop = -1; // reset
    char s[MAX * 2];
    strncpy(s, prefix, MAX * 2 - 1);
    s[MAX * 2 - 1] = '\0';

    char *token = strtok(s, " ");
    char *tokens[MAX];
    int tokenCount = 0;

    // Split into tokens
    while (token != NULL) {
        tokens[tokenCount++] = token;
        token = strtok(NULL, " ");
    }

```

```

}

// Evaluate from right to left
for (int i = tokenCount - 1; i >= 0; --i) {
    char *ch = tokens[i];

    if (isdigit(ch[0])) {
        pushEval(atof(ch));
    } else if (isalpha(ch[0])) {
        double v;
        printf("Enter value for %c: ", ch[0]);
        if (scanf("%lf", &v) != 1) { v = 0.0; }
        pushEval(v);
    } else if (ch[0] == '#') { // Unary minus
        double a = popEval();
        pushEval(-a);
    } else if (isOperator(ch[0])) {
        double a = popEval();
        double b = popEval();
        double res = 0.0;
        switch (ch[0]) {
            case '+': res = a + b; break;
            case '-': res = a - b; break;
            case '*': res = a * b; break;
            case '/':
                if (b == 0.0) {
                    printf("Error: division by zero.
Returning 0.\n");
                }
            }
        }
    }
}

```



```

        res = 0.0;
    } else res = a / b;

    break;

    case '^': res = pow(a, b); break;
}

pushEval(res);
}

}

if (evalTop >= 0) return popEval();
return 0.0;
}

int main() {
    char infix[MAX] = "";
    char prefix[MAX * 2];
    int choice;

    while (1) {
        printf("\n===== INFIX -> PREFIX MENU =====\n");
        printf("1. Input Infix Expression\n");
        printf("2. Convert to Prefix\n");
        printf("3. Evaluate Prefix\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1) break;
        getchar(); // eat newline
    }
}

```

```

switch (choice) {
    case 1:
        printf("Enter infix expression: ");
        if (fgets(infix, MAX, stdin) != NULL) {
            infix[strcspn(infix, "\n")] = '\0';
        }
        break;

    case 2:
        if (strlen(infix) == 0) {
            printf("Please input an infix
expression first (option 1).\n");
        } else {
            infixToPrefix(infix, prefix);
            printf("Prefix: %s\n", prefix);
        }
        break;

    case 3:
        if (strlen(infix) == 0) {
            printf("Please input an infix
expression first (option 1).\n");
        } else {
            infixToPrefix(infix, prefix);
            printf("Prefix: %s\n", prefix);
            double ans = evaluatePrefix(prefix);
            printf("Result: %.6g\n", ans);
        }
    }
}

```

```
        getchar();
    }
    break;

case 4:
    printf("Exiting...\n");
    exit(0);

default:
    printf("Invalid choice.\n");
}

}

return 0;
}
```

Output :

```
===== INFIX -> PREFIX MENU =====
1. Input Infix Expression
2. Convert to Prefix
3. Evaluate Prefix
4. Exit
Enter choice: 1
Enter infix expression: ((A+B)*C)-(D/E)

===== INFIX -> PREFIX MENU =====
1. Input Infix Expression
2. Convert to Prefix
3. Evaluate Prefix
4. Exit
Enter choice: 2
Prefix: - * + A B C / D E

===== INFIX -> PREFIX MENU =====
1. Input Infix Expression
2. Convert to Prefix
3. Evaluate Prefix
4. Exit
Enter choice: 3
Prefix: - * + A B C / D E
Enter value for E: 2
Enter value for D: 8
Enter value for C: 4
Enter value for B: 3
Enter value for A: 2
Result: 16
```